>_ MCMP113 · Week 10

Array Operations: Searching, Sorting, Max/Min

Maximum and minimum with index, statistics, linear and binary search, bubble, selection and insertion sort, counting the work

Asst. Prof. Hasan Oğuz, PhD
Introduction to Basic Algorithms and Programming · Istanbul Okan University
Vocational School · 2026–2027 Fall

How to use these notes

These notes follow the Week 10 slides and add what does not fit on a slide: the reasoning behind each algorithm, trace tables, the counts of comparisons and swaps, all thirteen example programs (complete, or as an excerpt for the long Example 13) with real sample runs, the common mistakes, a glossary and graded exercises. Every listing is read from the tested program in the `code` folder (`10_01` … `10_13`); open it in VS Code and run it while you read. Typed input in the sample runs is shown in **amber**. We still write everything as top-level statements: own methods come in Week 12, and we never use LINQ (`.Max()`, `.OrderBy()`, …). Bubble-sort pass tables and binary-search traces are typical final-exam questions.

Contents

1 Learning outcomes **2**

2 Maximum and minimum **2**

2.1 The champion pattern . . . 3
2.2 Example 1 – warmest and coldest day . . . 3
2.3 Example 2 – the second largest value . . . 4

3 Statistics with arrays **5**

3.1 Example 3 – grade statistics . . . 5

4 Linear search **7**

4.1 Example 4 – first index, all positions, count . . . 7

5 Binary search **8**

5.1 Example 5 – binary search with a printed trace . . . 9

6 Swapping and bubble sort **10**

6.1 Swapping two elements . . . 10
6.2 The idea of bubble sort . . . 10
6.3 Example 6 – bubble sort in C# . . . 11

7 Selection sort and insertion sort **12**

7.1 Selection sort . . . 12
7.2 Example 7 – selection sort in C# . . . 12
7.3 Example 8 – insertion sort (optional) . . . 13

8 Counting the work **14**

9 Built-in helpers and more array algorithms **15**
9.1 Example 9 – the Array class . . . 15
9.2 Example 10 – reverse in place and rotate . . . 16
9.3 Example 11 – remove duplicates from a sorted array . . . 17
9.4 Example 12 – merge two sorted arrays ⋆⋆⋆ . . . 18

10 Common mistakes **19**

11 Summary **19**

12 Glossary (English – Turkish) **20**

13 Exercises **21**
13.1 Check yourself: predict the output . . . 21
13.2 In-class exercises (pairs) . . . 21
13.3 Homework . . . 21
13.4 Extra practice . . . 22

14 References **22**

>_ 1 Learning outcomes

After this week you should be able to:

1. find the maximum and the minimum of an array *together with their index*, and the second largest value, and explain why the search must start with `a[0]` and not with 0;
2. compute the sum, the mean, the variance and the standard deviation of an array, and count the elements that satisfy a condition;
3. search an array linearly (first index, all positions, count) and use -1 as "not found";
4. search a *sorted* array with binary search and trace `low`, `mid`, `high` step by step;
5. swap two array elements, and sort an array with bubble sort (with early exit), selection sort and insertion sort; write the pass-by-pass table;
6. count the comparisons of these algorithms (n, $\log_2 n$, $n^2/2$) and use `Array.Sort`, `Array.IndexOf`, `Array.BinarySearch` and `Array.Reverse`;
7. reverse and rotate an array, remove duplicates from a sorted array and merge two sorted arrays.

>_ 2 Maximum and minimum

2.1 The champion pattern

To find the largest value we walk through the array once and remember the best value seen so far — the **champion**. The first element is the first champion; every later element that is *greater* replaces it. Because questions usually ask *where* the largest value is ("on which day was it warmest?"), we remember its **index** as well; the value itself is then always `a[maxIndex]`.

Maximum with index

$max \leftarrow a[0]$
$maxIndex \leftarrow 0$
FOR $i \leftarrow 1$ **TO** $n - 1$
  IF $a[i] > max$ **THEN**
    $max \leftarrow a[i]$
    $maxIndex \leftarrow i$
  END IF
END FOR
PRINT $max, maxIndex$

Trace for `temps` $= \{12.5, 18, 9.5, 21, 15\}$:

i	temps[i]	$>$ max?	max	maxIndex
–	–	start	12.5	0
1	18	true	18	1
2	9.5	false	18	1
3	21	true	21	3
4	15	false	21	3

Result: maximum 21 at index 3 (day 4).

The loop makes exactly $n - 1$ comparisons. For the minimum, replace $>$ by $<$. With $>$ the *first* of several equal maxima is kept; with $\geq$ the last one.

2.2 Example 1 – warmest and coldest day

Listing 1: 10_01_MaxMinIndex/Program.cs

```
// MCMP113 · Week 10 · Example 1 — Maximum and minimum with their index
// Start from the first element (not from 0) and remember WHERE the extreme value is.

Console.Write("How many days? ");
int n = int.Parse(Console.ReadLine());
double[] temps = new double[n];
for (int i = 0; i < n; i++)
{
    Console.Write($"Temperature of day {i + 1}: ");
    temps[i] = double.Parse(Console.ReadLine());
}

double max = temps[0];       // first element = first candidate
double min = temps[0];
int maxIndex = 0;
int minIndex = 0;
for (int i = 1; i < n; i++)  // temps[0] is already used
{
    if (temps[i] > max)
    {
        max = temps[i];
        maxIndex = i;
    }
    if (temps[i] < min)
    {
        min = temps[i];
        minIndex = i;
    }
}

Console.WriteLine($"Warmest: {max} on day {maxIndex + 1} (index {maxIndex})");
Console.WriteLine($"Coldest: {min} on day {minIndex + 1} (index {minIndex})");
```

```
33
34 // The classic mistake: starting the maximum at 0
35 double wrongMax = 0;
36 for (int i = 0; i < n; i++)
37 {
38     if (temps[i] > wrongMax)
39     {
40         wrongMax = temps[i];
41     }
42 }
43 Console.WriteLine($"Max if we start at 0: {wrongMax}");
```

Sample run 1 – a spring week

```
How many days? 5
Temperature of day 1: 12.5
Temperature of day 2: 18
Temperature of day 3: 9.5
Temperature of day 4: 21
Temperature of day 5: 15
Warmest: 21 on day 4 (index 3)
Coldest: 9.5 on day 3 (index 2)
Max if we start at 0: 21
```

Sample run 2 – a cold week

```
How many days? 4
Temperature of day 1: -3.5
Temperature of day 2: -1
Temperature of day 3: -7.25
Temperature of day 4: -2
Warmest: -1 on day 2 (index 1)
Coldest: -7.25 on day 3 (index 2)
Max if we start at 0: 0
```

Two separate `if` statements (not `else if`) are used, because one element is tested for both the maximum and the minimum. The last loop shows the classic mistake: if the maximum starts at 0 and all values are negative, no element is "> 0" and the program reports 0 — a temperature that is not even in the data (run 2). Start with `a[0]` (best: it also gives the index) or with `double.MinValue`, and always test with negative data.

2.3 Example 2 – the second largest value

Two champions: when a new largest value appears, the old one moves down to second place; otherwise the value may still beat the runner-up. A value equal to the largest is ignored, so "second largest" means the second largest *different* value. The flag `hasSecond` covers the case where all values are equal.

Listing 2: 10_02_SecondLargest/Program.cs

```
1  // MCMP113 · Week 10 · Example 2 — Second largest value in one pass
2  // Two "champions": when a new largest appears, the old largest becomes second.
3
4  int[] scores = { 72, 85, 64, 85, 90, 78 };
5  Console.WriteLine("Scores: " + string.Join(", ", scores));
6
7  int largest = scores[0];
8  int second = 0;
9  bool hasSecond = false;     // no second value seen yet
10 for (int i = 1; i < scores.Length; i++)
11 {
12     if (scores[i] > largest)
13     {
14         second = largest;   // old champion moves down
15         hasSecond = true;
16         largest = scores[i];
17     }
18     else if (scores[i] < largest && (!hasSecond || scores[i] > second))
19     {
20         second = scores[i];  // beats only the runner-up
21         hasSecond = true;
```

```
    }
    // scores[i] == largest: a duplicate, ignore it
}

Console.WriteLine($"Largest: {largest}");
if (hasSecond)
{
    Console.WriteLine($"Second largest: {second}");
}
else
{
    Console.WriteLine("No second largest: all values equal.");
}
```

i	scores[i]	case	largest	second
–	72	start	72	–
1	85	new largest	85	72
2	64	too small	85	72
3	85	duplicate	85	72
4	90	new largest	90	85
5	78	too small	90	85

Output

```
Scores: 72, 85, 64, 85, 90, 78
Largest: 90
Second largest: 85
```

Sorting and taking `a[n - 2]` is *not* a correct shortcut: for $\{90, 90, 85\}$ it returns 90.

>_ 3 Statistics with arrays

For n values $x_1, \ldots, x_n$ (in C#: `a[0]` ... `a[n - 1]`):

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad \sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2, \qquad \sigma = \sqrt{\sigma^2}.$$

The **mean** $\bar{x}$ needs one pass. The **variance** σ^2 needs a *second* pass, because the distances are measured from the mean, which is known only after the first pass — this is why the values must be stored in an array. The **standard deviation** σ has the same unit as the data. (Statistics books often divide by $n - 1$ instead of n — the *sample* standard deviation; in this course we divide by n.)

x_i	$x_i - \bar{x}$	$(x_i - \bar{x})^2$
72	5.67	32.11
45	−21.33	455.11
88	21.67	469.44
60	−6.33	40.11
95	28.67	821.78
38	−28.33	802.78
$\Sigma = 398$	$\bar{x} = 66.33$	2621.33

$\sigma^2 = 2621.33/6 = 436.89$
$\sigma = \sqrt{436.89} \approx 20.90$ points
(sample s.d. with $n - 1$: 22.90)

3.1 Example 3 – grade statistics

The third pattern of the week is **counting with a condition**: a counter that grows only when an `if` is true. Several counters can share one loop.

Listing 3: 10_03_GradeStatistics/Program.cs

```
// MCMP113 · Week 10 · Example 3 — Grade statistics: mean, standard deviation, counting
// Two passes: first the mean, then the squared distances from the mean.

Console.Write("Number of students: ");
int n = int.Parse(Console.ReadLine());
double[] grades = new double[n];
for (int i = 0; i < n; i++)
{
    Console.Write($"Grade {i + 1}: ");
    grades[i] = double.Parse(Console.ReadLine());
}

// pass 1: sum and mean
double sum = 0;
for (int i = 0; i < n; i++)
{
    sum += grades[i];
}
double mean = sum / n;

// pass 2: variance = average of squared distances
double squares = 0;
for (int i = 0; i < n; i++)
{
    double d = grades[i] - mean;
    squares += d * d;
}
double variance = squares / n;
double sigma = Math.Sqrt(variance);

// counting with a condition
int passed = 0;
int aboveMean = 0;
for (int i = 0; i < n; i++)
{
    if (grades[i] >= 50)
    {
        passed++;
    }
    if (grades[i] > mean)
    {
        aboveMean++;
    }
}

Console.WriteLine($"Sum      = {sum}");
Console.WriteLine($"Mean     = {mean:F2}");
Console.WriteLine($"Variance = {variance:F2}");
Console.WriteLine($"Std dev  = {sigma:F2}");
Console.WriteLine($"Passed (>= 50): {passed} of {n}");
Console.WriteLine($"Above the mean: {aboveMean} of {n}");
```

Sample run

```
Number of students: 6
Grade 1: 72
Grade 2: 45
Grade 3: 88
Grade 4: 60
Grade 5: 95
Grade 6: 38
Sum      = 398
Mean     = 66.33
Variance = 436.89
Std dev  = 20.90
Passed (>= 50): 4 of 6
Above the mean: 3 of 6
```

>_ 4 Linear search

Linear (sequential) **search** compares the target with `a[0]`, `a[1]`, ... until it is found or the array ends. It works on *any* array. If only the first occurrence is needed, the loop stops early; it then has two exits, exactly like the password loop of Week 7. The index -1 is the conventional answer "not found": no array has an element with index -1 (`Array.IndexOf` uses the same convention). Best case: 1 comparison; worst case (last element, or not present): n comparisons.

Linear search, first occurrence

$found \leftarrow$ false; $index \leftarrow -1$; $i \leftarrow 0$
WHILE $i < n$ **AND NOT** $found$
  IF $a[i] = target$ **THEN**
    $found \leftarrow$ true
    $index \leftarrow i$
  END IF
  $i \leftarrow i + 1$
END WHILE

Search 78: found after 2 checks

45	78	92	60	78	55	83	78

Search 50: not found after 8 checks

45	78	92	60	78	55	83	78

amber: checked, no match; green: match

4.1 Example 4 – first index, all positions, count

Part 1 stops early (`while` with a flag); part 2 must visit *every* element, so a `for` loop without early exit is the right tool. The variable `comparisons` is an *instrument*: it measures the work without changing the result. (Part 2 uses `k`, because `i` is already declared at the top level.)

Listing 4: 10_04_LinearSearch/Program.cs

```
1  // MCMP113 · Week 10 · Example 4 — Linear search: first index, all positions, count
2  // Look at the elements one by one; stop early when only the first match is needed.
3
4  int[] scores = { 45, 78, 92, 60, 78, 55, 83, 78 };
5  Console.WriteLine("Scores: " + string.Join(" ", scores));
6  Console.Write("Search for: ");
7  int target = int.Parse(Console.ReadLine());
8
9  // 1) first occurrence: stop as soon as it is found
10 int index = -1;          // -1 means "not found"
11 bool found = false;
12 int comparisons = 0;
13 int i = 0;
14 while (i < scores.Length && !found)
```

```
{
    comparisons++;
    if (scores[i] == target)
    {
        found = true;
        index = i;
    }
    i++;
}

if (found)
{
    Console.WriteLine($"First at index {index}: {comparisons} checks");
}
else
{
    Console.WriteLine($"Not found: {comparisons} checks");
}

// 2) all positions and the count: check EVERY element
int count = 0;
Console.Write("All positions:");
for (int k = 0; k < scores.Length; k++)
{
    if (scores[k] == target)
    {
        Console.Write($" {k}");
        count++;
    }
}
Console.WriteLine();
Console.WriteLine($"Count: {count}");
```

Sample run 1

```
Scores: 45 78 92 60 78 55 83 78
Search for: 78
First at index 1: 2 checks
All positions: 1 4 7
Count: 3
```

Sample run 2

```
Scores: 45 78 92 60 78 55 83 78
Search for: 50
Not found: 8 checks
All positions:
Count: 0
```

>_ 5 Binary search

In Weeks 1 and 7 we guessed a secret number between 1 and 100 by always guessing the middle: every answer "higher" or "lower" throws away half of the range, so 7 guesses are always enough. **Binary search** plays the same game on the indices of a **sorted** array. `low` and `high` mark the part of the array that can still contain the target, and `mid` = (`low` + `high`) `/ 2` (integer division) is the guess.

Binary search needs sorted data

Only in a sorted array does `a[mid] < target` prove that the target lies to the *right* of `mid`. On unsorted data binary search silently gives wrong answers.

Binary search

$low \leftarrow 0;\ high \leftarrow n-1;\ index \leftarrow -1$
WHILE $low \le high$ **AND** $index = -1$
  $mid \leftarrow (low + high)$ div 2
  IF $a[mid] = target$ **THEN**
    $index \leftarrow mid$
  ELSE
    IF $a[mid] < target$ **THEN**
      $low \leftarrow mid + 1$ (right)
    ELSE
      $high \leftarrow mid - 1$ (left)
    END IF
  END IF
END WHILE

Trace: search 20 in
`4 9 13 18 22 27 31 36 40 45 51 58 63 70 77`

step	low	high	mid	a[mid]	decision
1	0	14	7	36	$36 > 20$: high $= 6$
2	0	6	3	18	$18 < 20$: low $= 4$
3	4	6	5	27	$27 > 20$: high $= 4$
4	4	4	4	22	$22 > 20$: high $= 3$

$\text{low} = 4 > \text{high} = 3$: empty range, not found

Each step halves the range: $15 \to 7 \to 3 \to 1$ elements. The worst case for n elements is $\lfloor \log_2 n \rfloor + 1$ comparisons: 4 for $n = 15$, 7 for 100, 10 for 1000, 20 for one million.

5.1 Example 5 – binary search with a printed trace

The program prints one line per step, so the output *is* the trace table.

Listing 5: 10_05_BinarySearch/Program.cs

```
// MCMP113 · Week 10 · Example 5 — Binary search on a sorted array (with a trace)
// Like guess-the-number: look at the middle and throw away half of the range.

// the data MUST be sorted
int[] a = { 4, 9, 13, 18, 22, 27, 31, 36, 40, 45, 51, 58, 63, 70, 77 };
Console.WriteLine("a: " + string.Join(" ", a));
Console.Write("Search for: ");
int target = int.Parse(Console.ReadLine());

int low = 0;
int high = a.Length - 1;
int index = -1;          // -1 = not found (yet)
int step = 0;
while (low <= high && index == -1)
{
    int mid = (low + high) / 2;          // integer division
    step++;
    Console.Write($"Step {step}: low={low} high={high} ");
    Console.WriteLine($"mid={mid} a[mid]={a[mid]}");
    if (a[mid] == target)
    {
        index = mid;
    }
    else if (a[mid] < target)
    {
        low = mid + 1;                   // go right
    }
    else
    {
        high = mid - 1;                  // go left
    }
}

if (index >= 0)
```

```
{
    Console.WriteLine($"Found at index {index}, steps: {step}");
}
else
{
    Console.WriteLine($"Not found (low > high), steps: {step}");
}
```

Sample runs: 58 (found in 2 steps), 20 (not found), 77 (last element, 4 steps)

```
a: 4 9 13 18 22 27 31 36 40 45 51 58 63 70 77
Search for: 58
Step 1: low=0 high=14 mid=7 a[mid]=36
Step 2: low=8 high=14 mid=11 a[mid]=58
Found at index 11, steps: 2

a: 4 9 13 18 22 27 31 36 40 45 51 58 63 70 77
Search for: 20
Step 1: low=0 high=14 mid=7 a[mid]=36
Step 2: low=0 high=6 mid=3 a[mid]=18
Step 3: low=4 high=6 mid=5 a[mid]=27
Step 4: low=4 high=4 mid=4 a[mid]=22
Not found (low > high), steps: 4

a: 4 9 13 18 22 27 31 36 40 45 51 58 63 70 77
Search for: 77
Step 1: low=0 high=14 mid=7 a[mid]=36
Step 2: low=8 high=14 mid=11 a[mid]=58
Step 3: low=12 high=14 mid=13 a[mid]=70
Step 4: low=14 high=14 mid=14 a[mid]=77
Found at index 14, steps: 4
```

>_ 6 Swapping and bubble sort

6.1 Swapping two elements

To exchange `a[i]` and `a[j]` we need a third "glass": `temp`. Without it, the first assignment overwrites one of the values (`a[i] = a[j]; a[j] = a[i];` leaves both equal). C# also offers the tuple swap `(a[i], a[j]) = (a[j], a[i]);` — correct and short; on exams, the `temp` version is expected.

```
int temp = a[i];   // 1) save a[i]
a[i] = a[j];       // 2) overwrite a[i]
a[j] = temp;       // 3) old a[i] into a[j]
```

6.2 The idea of bubble sort

Bubble sort compares *neighbours* `a[j]` and `a[j + 1]` and swaps them if they are in the wrong order. One **pass** walks from left to right. After pass 1 the largest value is at the end (it "bubbles up"); after pass p the last p cells are final, so pass p needs only $n - p$ comparisons. If a whole pass makes *no* swap, all neighbours are in order and the array is sorted: the flag `swapped` lets us stop early (**early exit**).

Bubble sort with early exit

FOR $pass \leftarrow 1$ **TO** $n-1$
  $swapped \leftarrow$ false
  FOR $j \leftarrow 0$ **TO** $n - pass - 1$
    IF $a[j] > a[j+1]$ **THEN**
      swap $a[j]$ and $a[j+1]$
      $swapped \leftarrow$ true
    END IF
  END FOR
  IF NOT $swapped$ **THEN**
    stop (sorted)
  END IF
END FOR

Pass 1 in detail (amber: compared pair)

$j = 0$	31	12	47	8	25	19	swap
$j = 1$	12	31	47	8	25	19	keep
$j = 2$	12	31	47	8	25	19	swap
$j = 3$	12	31	8	47	25	19	swap
$j = 4$	12	31	8	25	47	19	swap
after	12	31	8	25	19	47	

All passes (green: final cells). This is exactly what Example 6 prints.

after	array						comparisons	swaps
start	31	12	47	8	25	19		
pass 1	12	31	8	25	19	47	5	4
pass 2	12	8	25	19	31	47	4	3
pass 3	8	12	19	25	31	47	3	2
pass 4	8	12	19	25	31	47	2	0 → stop
total							14	9

The array is already sorted after pass 3, but the algorithm only *knows* it after pass 4, which makes no swap. Without the early exit, pass 5 would add one more comparison ($15 = 5+4+3+2+1$). For an already sorted array bubble sort needs a single pass ($n-1$ comparisons, no swap); for a reversed array it needs all $n(n-1)/2$ comparisons and as many swaps.

6.3 Example 6 – bubble sort in C#

Listing 6: 10_06_BubbleSort/Program.cs

```
// MCMP113 · Week 10 · Example 6 – Bubble sort with early exit, pass by pass
// Neighbours that are in the wrong order are swapped; the largest value "bubbles" to the end.

int[] a = { 31, 12, 47, 8, 25, 19 };
int n = a.Length;
Console.WriteLine("Start:  " + string.Join(" ", a));

int comparisons = 0;
int swaps = 0;
for (int pass = 1; pass <= n - 1; pass++)
{
    bool swapped = false;              // no swap yet
    for (int j = 0; j < n - pass; j++) // tail is sorted
    {
        comparisons++;
        if (a[j] > a[j + 1])
        {
            int temp = a[j];           // swap via temp
            a[j] = a[j + 1];
            a[j + 1] = temp;
            swapped = true;
```

```
            swaps++;
        }
    }
    Console.WriteLine($"Pass {pass}: " + string.Join(" ", a));
    if (!swapped)                       // sorted: stop early
    {
        break;
    }
}

Console.WriteLine("Sorted: " + string.Join(" ", a));
Console.WriteLine($"Comparisons: {comparisons}, swaps: {swaps}");
```

Output

```
Start:  31 12 47 8 25 19
Pass 1: 12 31 8 25 19 47
Pass 2: 12 8 25 19 31 47
Pass 3: 8 12 19 25 31 47
Pass 4: 8 12 19 25 31 47
Sorted: 8 12 19 25 31 47
Comparisons: 14, swaps: 9
```

The inner condition is `j < n - pass`. Writing `j < n` makes `a[j + 1]` equal to `a[n]` in the last step: `IndexOutOfRangeException`. For descending order change > into <.

>_ 7 Selection sort and insertion sort

7.1 Selection sort

Selection sort *selects* the smallest element of the unsorted part `a[i..n-1]` — the "minimum with index" of Section 2 — and swaps it to position i. There is at most *one* swap per pass (at most $n-1$ in total), but always $(n-1)+(n-2)+\cdots+1 = n(n-1)/2$ comparisons, because there is no early exit. For $n = 6$: 15 comparisons.

Selection sort

FOR $i \leftarrow 0$ **TO** $n-2$
  $minIndex \leftarrow i$
  FOR $j \leftarrow i+1$ **TO** $n-1$
    IF $a[j] < a[minIndex]$ **THEN**
      $minIndex \leftarrow j$
    END IF
  END FOR
  IF $minIndex \neq i$ **THEN**
    swap $a[i]$ and $a[minIndex]$
  END IF
END FOR

Array *before* the swap of pass i
(blue: minimum, amber: position i, green: final)

$i = 0$	31	12	47	8	25	19	$31 \leftrightarrow 8$
$i = 1$	8	12	47	31	25	19	no swap
$i = 2$	8	12	47	31	25	19	$47 \leftrightarrow 19$
$i = 3$	8	12	19	31	25	47	$31 \leftrightarrow 25$
$i = 4$	8	12	19	25	31	47	no swap

7.2 Example 7 – selection sort in C#

Listing 7: 10_07_SelectionSort/Program.cs

```
// MCMP113 · Week 10 · Example 7 — Selection sort, pass by pass
// Find the smallest element of the unsorted part and put it at the front with ONE swap.
```

```
3
4  int[] a = { 31, 12, 47, 8, 25, 19 };
5  int n = a.Length;
6  Console.WriteLine("Start:  " + string.Join(" ", a));
7
8  int comparisons = 0;
9  int swaps = 0;
10 for (int i = 0; i < n - 1; i++)
11 {
12     int minIndex = i;          // where the smallest is
13     for (int j = i + 1; j < n; j++)
14     {
15         comparisons++;
16         if (a[j] < a[minIndex])
17         {
18             minIndex = j;
19         }
20     }
21     if (minIndex != i)         // at most one swap per pass
22     {
23         int temp = a[i];
24         a[i] = a[minIndex];
25         a[minIndex] = temp;
26         swaps++;
27     }
28     Console.Write($"Pass {i + 1}: " + string.Join(" ", a));
29     Console.WriteLine($"   (min = {a[i]})");
30 }
31
32 Console.WriteLine("Sorted: " + string.Join(" ", a));
33 Console.WriteLine($"Comparisons: {comparisons}, swaps: {swaps}");
```

Output

```
Start:  31 12 47 8 25 19
Pass 1: 8 12 47 31 25 19   (min = 8)
Pass 2: 8 12 47 31 25 19   (min = 12)
Pass 3: 8 12 19 31 25 47   (min = 19)
Pass 4: 8 12 19 25 31 47   (min = 25)
Pass 5: 8 12 19 25 31 47   (min = 31)
Sorted: 8 12 19 25 31 47
Comparisons: 15, swaps: 3
```

7.3 Example 8 – insertion sort (optional)

Insertion sort works like sorting playing cards in your hand: the left part `a[0..i-1]` is always sorted; take the next card `key = a[i]`, shift every bigger card one place to the right and drop the key into the gap. On almost-sorted data it needs only about n comparisons.

Listing 8: 10_08_InsertionSort/Program.cs

```
1  // MCMP113 · Week 10 · Example 8 — Insertion sort (optional), pass by pass
2  // Like sorting playing cards in your hand: take the next card, shift bigger cards right, insert.
3
4  int[] a = { 31, 12, 47, 8, 25, 19 };
5  int n = a.Length;
6  Console.WriteLine("Start:  " + string.Join(" ", a));
7
8  int comparisons = 0;
9  int shifts = 0;
10 for (int i = 1; i < n; i++)
11 {
```

```
    int key = a[i];            // the "card" to insert
    int j = i - 1;
    while (j >= 0)
    {
        comparisons++;
        if (a[j] > key)
        {
            a[j + 1] = a[j];   // shift one cell right
            shifts++;
            j--;
        }
        else
        {
            break;             // found the place
        }
    }
    a[j + 1] = key;
    Console.Write($"Pass {i}: " + string.Join(" ", a));
    Console.WriteLine($"   ({key} -> [{j + 1}])");
}

Console.WriteLine("Sorted: " + string.Join(" ", a));
Console.WriteLine($"Comparisons: {comparisons}, shifts: {shifts}");
```

Output

```
Start:  31 12 47 8 25 19
Pass 1: 12 31 47 8 25 19   (12 -> [0])
Pass 2: 12 31 47 8 25 19   (47 -> [2])
Pass 3: 8 12 31 47 25 19   (8 -> [0])
Pass 4: 8 12 25 31 47 19   (25 -> [2])
Pass 5: 8 12 19 25 31 47   (19 -> [2])
Sorted: 8 12 19 25 31 47
Comparisons: 12, shifts: 9
```

same 6 numbers	comparisons	swaps / shifts	idea
bubble sort	14	9 swaps	swap neighbours; stop after a pass without swaps
selection sort	15	3 swaps	select the minimum, one swap per pass
insertion sort	12	9 shifts	insert the next value into the sorted left part

>_ 8 Counting the work

How does the work grow when the array grows? Example 13 fills arrays of $n = 10, 100, 1000, 10\,000$ random numbers (with the fixed seed `new Random(2026)`, so the counts are the same on every run), searches a value that is not in the data (the worst case) and sorts copies of the data with bubble and selection sort, counting comparisons and swaps. The listing shows the beginning of the program; the selection sort and binary search parts are the code of Examples 7 and 5 with counters.

Listing 9: 10_13_SortingRace/Program.cs (lines 1–12)

```
// MCMP113 · Week 10 · Example 13 — Counting the work: search and sort for growing n
// Instrumented loops count comparisons; a fixed Random seed gives the same data on every run.

int[] sizes = { 10, 100, 1000, 10000 };
Random rng = new Random(2026);  // fixed seed: same data each run
int target = 1000000;           // not in the data (0..999999)

Console.Write("     n  linear  binary  ");
Console.WriteLine("bubble-cmp  bubble-swp  select-cmp  select-swp  bubble-ms");
```

```
10 for (int s = 0; s < sizes.Length; s++)
11 {
12     int n = sizes[s];
```

Listing 10: 10_13_SortingRace/Program.cs (lines 30–55)

```
30     // bubble sort (with early exit) on a copy
31     int[] b = (int[])data.Clone();
32     long bubbleCmp = 0;
33     long bubbleSwp = 0;
34     var watch = System.Diagnostics.Stopwatch.StartNew();
35     for (int pass = 1; pass <= n - 1; pass++)
36     {
37         bool swapped = false;
38         for (int j = 0; j < n - pass; j++)
39         {
40             bubbleCmp++;
41             if (b[j] > b[j + 1])
42             {
43                 int temp = b[j];
44                 b[j] = b[j + 1];
45                 b[j + 1] = temp;
46                 bubbleSwp++;
47                 swapped = true;
48             }
49         }
50         if (!swapped)
51         {
52             break;
53         }
54     }
55     watch.Stop();
```

Output (the last column is a measured time and differs between runs and PCs)

```
     n  linear  binary  bubble-cmp  bubble-swp  select-cmp  select-swp  bubble-ms
    10      10       4          42          24          45           8          0
   100     100       7        4935        2162        4950          94          0
  1000    1000      10      497484      244419      499500         991          2
 10000   10000      14    49994565    25154921    49995000        9989        290
Worst case of both sorts: n(n-1)/2 comparisons.
```

- **Linear search** needs n comparisons in the worst case: $10\times$ more data, $10\times$ more work. We write $O(n)$ ("order n").
- **Binary search** needs $\lfloor \log_2 n \rfloor + 1$: doubling n adds only one step — $O(\log n)$.
- **Selection sort** always makes $n(n-1)/2$ comparisons $(45, 4950, 499\,500, 49\,995\,000)$; bubble sort makes slightly fewer thanks to the early exit. $10\times$ more data means about $100\times$ more work: $O(n^2)$. With $n = 10\,000$ this takes a noticeable fraction of a second.
- `Array.Sort` uses a much faster method ($O(n \log n)$, introsort). Lesson: learn the simple sorts to understand the ideas, but call `Array.Sort` in real programs. If you search an array many times, sort it once and use binary search.

>_ 9 Built-in helpers and more array algorithms

9.1 Example 9 – the Array class

Listing 11: 10_09_ArrayBuiltIns/Program.cs

```
// MCMP113 · Week 10 · Example 9 — Built-in helpers of the Array class
// What professionals use: Array.Sort, Array.Reverse, Array.IndexOf, Array.BinarySearch.

int[] a = { 31, 12, 47, 8, 25, 19 };
Console.WriteLine("Original:      " + string.Join(" ", a));

Console.WriteLine($"IndexOf(25):   {Array.IndexOf(a, 25)}");
Console.WriteLine($"IndexOf(99):   {Array.IndexOf(a, 99)}"); // -1

Array.Sort(a);                    // ascending, in place
Console.WriteLine("After Sort:    " + string.Join(" ", a));

Console.WriteLine($"BinarySearch(25): {Array.BinarySearch(a, 25)}");
int pos = Array.BinarySearch(a, 20); // < 0: not found
Console.WriteLine($"BinarySearch(20): {pos}  (insert at {-pos - 1})");

Array.Reverse(a);                 // now descending
Console.WriteLine("After Reverse: " + string.Join(" ", a));

string[] names = { "Zeynep", "ali", "Can", "Deniz", "Emre" };
Array.Sort(names);  // our template: ordinal, capitals first
Console.WriteLine("Ordinal:     " + string.Join(", ", names));
Array.Sort(names, StringComparer.OrdinalIgnoreCase);
Console.WriteLine("Ignore case: " + string.Join(", ", names));
```

Output

```
Original:      31 12 47 8 25 19
IndexOf(25):   4
IndexOf(99):   -1
After Sort:    8 12 19 25 31 47
BinarySearch(25): 3
BinarySearch(20): -4  (insert at 3)
After Reverse: 47 31 25 19 12 8
Ordinal:     Can, Deniz, Emre, Zeynep, ali
Ignore case: ali, Can, Deniz, Emre, Zeynep
```

`Array.BinarySearch` requires a sorted array. A negative result r means "not found"; $-r-1$ is the index where the value would have to be inserted to keep the order ($-(-4)-1=3$). Our project template switches on `InvariantGlobalization`, so strings are sorted by character code (capitals before lower-case letters); `StringComparer.OrdinalIgnoreCase` ignores the case. There is no `Array.Max`: `a.Max()` is LINQ, which we do not use — write the loop.

9.2 Example 10 – reverse in place and rotate

Reversing *in place* uses two indices that start at both ends and walk towards each other, swapping as they go; for 7 elements that is 3 swaps and the middle element stays. (A `for` loop over *all* indices that swaps `a[i]` with `a[n-1-i]` swaps every pair twice and leaves the array unchanged.) Rotating left by k moves the element at index i to index $(i-k+n) \bmod n$; adding n keeps the left operand of `%` non-negative (Week 4), and k is first reduced modulo n, so $k=9$ on 7 elements equals $k=2$.

Listing 12: 10_10_ReverseRotate/Program.cs

```
// MCMP113 · Week 10 · Example 10 — Reverse in place and rotate an array
// Two indices walk towards each other; rotation uses a second array and the remainder %.

int[] a = { 1, 2, 3, 4, 5, 6, 7 };
Console.WriteLine("Original:       " + string.Join(" ", a));

```

```
// reverse in place: swap the outer pair, move inwards
int left = 0;
int right = a.Length - 1;
while (left < right)
{
    int temp = a[left];
    a[left] = a[right];
    a[right] = temp;
    // one-line tuple swap (same effect):
    // (a[left], a[right]) = (a[right], a[left]);
    left++;
    right--;
}
Console.WriteLine("Reversed:       " + string.Join(" ", a));

// rotate LEFT by k: element i moves to (i - k + n) % n
int[] b = { 1, 2, 3, 4, 5, 6, 7 };
int n = b.Length;
Console.Write("Rotate left by k = ");
int k = int.Parse(Console.ReadLine()) % n; // k = n: no change
int[] rotated = new int[n];
for (int i = 0; i < n; i++)
{
    rotated[(i - k + n) % n] = b[i];
}
Console.WriteLine("Rotated left:   " + string.Join(" ", rotated));
```

Sample run 1

```
Original:       1 2 3 4 5 6 7
Reversed:       7 6 5 4 3 2 1
Rotate left by k = 3
Rotated left:   4 5 6 7 1 2 3
```

Sample run 2

```
Original:       1 2 3 4 5 6 7
Reversed:       7 6 5 4 3 2 1
Rotate left by k = 9
Rotated left:   3 4 5 6 7 1 2
```

9.3 Example 11 – remove duplicates from a sorted array

In a sorted array equal values are neighbours. The write index `unique` marks the end of the kept part `a[0..unique-1]`; a value is copied only if it differs from the last kept value. The read index `i` is always ahead of `unique`, so nothing unread is overwritten. Arrays cannot shrink, so the kept part is finally copied into a new, shorter array.

Listing 13: 10_11_RemoveDuplicates/Program.cs

```
// MCMP113 · Week 10 · Example 11 — Remove duplicates from a SORTED array
// In a sorted array equal values are neighbours: keep a value only if it differs from the last
    kept one.

int[] a = { 2, 2, 3, 5, 5, 5, 8, 9, 9 };
Console.WriteLine("Sorted input: " + string.Join(" ", a));

int unique = 1;                 // a[0] is always kept
for (int i = 1; i < a.Length; i++)
{
    if (a[i] != a[unique - 1])  // new value?
    {
        a[unique] = a[i];       // next free cell
        unique++;
    }
}

int[] result = new int[unique]; // keep first cells
```

```
for (int i = 0; i < unique; i++)
{
    result[i] = a[i];
}
Console.WriteLine($"Unique ({unique}): " + string.Join(" ", result));
Console.WriteLine($"Removed duplicates: {a.Length - unique}");
```

Output

```
Sorted input: 2 2 3 5 5 5 8 9 9
Unique (5): 2 3 5 8 9
Removed duplicates: 4
```

9.4 Example 12 – merge two sorted arrays ⋆⋆⋆

Three indices: `i` reads `a`, `j` reads `b`, `k` writes `c`. In each step the smaller of the two front elements is copied and only *its* index advances. When one array is exhausted, the rest of the other one is copied. Trace: 3 vs 1 → 1; 3 vs 9 → 3; 8 vs 9 → 8; 15 vs 9 → 9; 15 vs 10 → 10; 15 vs 22 → 15; 21 vs 22 → 21; then 22, 30 are copied. This merge step is the heart of *merge sort*, an $O(n \log n)$ algorithm.

Listing 14: 10_12_MergeSorted/Program.cs

```
// MCMP113 · Week 10 · Example 12 — Merge two sorted arrays into one sorted array
// Compare the two front elements, copy the smaller one, advance in that array.

int[] a = { 3, 8, 15, 21 };
int[] b = { 1, 9, 10, 22, 30 };
int[] c = new int[a.Length + b.Length];
Console.WriteLine("a: " + string.Join(" ", a));
Console.WriteLine("b: " + string.Join(" ", b));

int i = 0;   // next unused index in a
int j = 0;   // next unused index in b
int k = 0;   // next free index in c
while (i < a.Length && j < b.Length)
{
    if (a[i] <= b[j])
    {
        c[k] = a[i];
        i++;
    }
    else
    {
        c[k] = b[j];
        j++;
    }
    k++;
}
while (i < a.Length)   // copy what is left of a
{
    c[k] = a[i];
    i++;
    k++;
}
while (j < b.Length)   // copy what is left of b
{
    c[k] = b[j];
    j++;
    k++;
}

Console.WriteLine("merged: " + string.Join(" ", c));
```

Output

```
a: 3 8 15 21
b: 1 9 10 22 30
merged: 1 3 8 9 10 15 21 22 30
```

>_ 10 Common mistakes

1. Starting the maximum at 0

Wrong for negative data (Example 1, run 2). Start with `a[0]` and loop from index 1.

2. Reading past the end

`for (int j = 0; j < n; j++)` with `a[j + 1]` throws `IndexOutOfRangeException`. Use `j < n - 1` (or `j < n - pass`).

3. Swap without temp

`a[i] = a[j]; a[j] = a[i];` loses the old `a[i]`. Use `temp` or the tuple swap.

4. Binary search on unsorted data, or wrong updates

Sort first. Update with `low = mid + 1` and `high = mid - 1`: `low = mid` can loop forever (e.g. `low = 3`, `high = 4`: `mid` stays 3).

5. 0 as "not found"

0 is a valid index. Use -1.

6. Sorting the original by accident

Sorting changes the array in place, and `int[] b = a;` is not a copy (Week 9). Sort a copy: `(int[])a.Clone()`.

>_ 11 Summary

- Max/min: start with `a[0]`, loop from 1, remember the index. Second largest: two champions.
- Mean in pass 1, squared distances in pass 2; $\sigma = \sqrt{\sigma^2}$. Counting: `count++` inside an `if`.
- Linear search: any array, up to n comparisons, -1 means "not found".
- Binary search: sorted arrays only; `low`, `mid`, `high`; at most $\lfloor \log_2 n \rfloor + 1$ comparisons.
- Bubble sort swaps neighbours and stops after a pass without swaps; selection sort selects the minimum and swaps at most once per pass; insertion sort inserts into a sorted left part. All need about $n^2/2$ comparisons in the worst case; `Array.Sort` is much faster.
- Two indices from both ends reverse an array; a write index removes duplicates; three indices merge two sorted arrays.

>_ 12 Glossary (English – Turkish)

English	Turkish	Meaning
maximum / minimum	en büyük / en küçük	largest / smallest value
index	indis	position of an element (from 0)
mean (average)	ortalama	sum divided by the number of values
variance	varyans	mean of the squared distances from the mean
standard deviation	standart sapma	square root of the variance
linear (sequential) search	doğrusal (sıralı) arama	check the elements one by one
binary search	ikili arama	halve a sorted range each step
sorted (ascending / descending)	sıralı (artan / azalan)	in increasing / decreasing order
sorting algorithm	sıralama algoritması	method that puts values in order
bubble sort	kabarcık sıralaması	swaps neighbours, largest goes last
selection sort	seçmeli sıralama	selects the minimum, one swap per pass
insertion sort	eklemeli sıralama	inserts into a sorted left part
swap	yer değiştirme	exchange two values (with `temp`)
pass	geçiş	one walk through the array
comparison	karşılaştırma	one test such as `a[j] > a[j + 1]`
in place	yerinde	without a second array
rotate	döndürme (kaydırma)	shift all elements cyclically
merge	birleştirme	combine two sorted arrays into one
duplicate	tekrar eden (yinelenen) değer	value that occurs more than once
time complexity, $O(\cdot)$	zaman karmaşıklığı	how the work grows with n

>_ 13 Exercises

13.1 Check yourself: predict the output

Write the exact output. Then run the code to check.

Q1

```
int[] a = { 5, 3, 9, 1, 7 };
int m = 0;
for (int i = 1; i < a.Length; i++)
{
    if (a[i] < a[m])
    {
        m = i;
    }
}
Console.WriteLine($"{m} {a[m]}");
```

Q2

```
int[] b = { 2, 5, 8, 12, 16,
            23, 38, 56, 72, 91 };
int low = 0;
int high = b.Length - 1;
while (low <= high)
{
    int mid = (low + high) / 2;
    Console.Write($"{mid} ");
    if (b[mid] == 23)
    {
        break;
    }
    else if (b[mid] < 23)
    {
        low = mid + 1;
    }
    else
    {
        high = mid - 1;
    }
}
```

Q3

```
int[] c = { 4, 2, 5, 1, 3 };
int swaps = 0;
for (int j = 0; j < c.Length - 1; j++)
{
    if (c[j] > c[j + 1])
    {
        int t = c[j];
        c[j] = c[j + 1];
        c[j + 1] = t;
        swaps++;
    }
}
string line = string.Join(" ", c);
Console.WriteLine($"{line} | {swaps}");
```

13.2 In-class exercises (pairs)

E1. ⋆ **Range of the week.** Read 7 temperatures into an array. Print the maximum and the minimum with their day numbers (1–7) and the range max − min.

E2. ⋆ **Find a student.** `string[] names = { "Ali", "Zeynep", "Can", "Deniz", "Emre", "Elif" };` Read a name and print its index, or "not found (−1)". Upper/lower case must not matter.

E3. ⋆⋆ **Descending order.** Change Example 6 so that it sorts from the largest to the smallest value. Print every pass and count passes, comparisons and swaps.

E4. ⋆⋆ **Every target.** Search each of the 15 values of Example 5 in turn with binary search. Print the number of steps for each value and the average.

13.3 Homework

H1. ⋆⋆ **Median.** Read n grades. Sort a *copy* (keep the original order), print both and the median: the middle element for odd n, the mean of the two middle elements for even n.

H2. ⋆ **Two smallest.** Print the smallest and the second smallest *different* value of an array, or a message if there is no second smallest.

H3. ⋆⋆ **Mode.** Sort an `int` array, then find the most frequent value in one pass by counting runs of equal neighbours.

H4. ⋆ **Is it sorted?** Check with one loop whether an array is sorted ascending; if not, print the first index where the order breaks.

13.4 Extra practice

P1. ⋆ Trace selection sort on $\{5, 1, 4, 2, 3\}$: write the array after every pass and count the swaps.

P2. ⋆ Trace insertion sort on the same array $\{5, 1, 4, 2, 3\}$.

P3. ⋆ What is the largest number of comparisons binary search needs for $n = 100$, 1000 and 1 000 000 elements? And linear search?

P4. ⋆⋆ How many comparisons and swaps does bubble sort with early exit make for the sorted array $\{1, 2, 3, 4, 5, 6\}$ and for the reversed array $\{6, 5, 4, 3, 2, 1\}$?

P5. ⋆⋆ Rotate $\{1, 2, 3, 4, 5, 6, 7\}$ to the *right* by $k = 2$ (expected: `6 7 1 2 3 4 5`). Which formula replaces $(i - k + n) \bmod n$?

P6. ⋆⋆⋆ Find the bug: a student writes `low = mid;` instead of `low = mid + 1;` in binary search. Search 77 in the array of Example 5 by hand. What happens?

>_ 14 References

- Microsoft Learn — Arrays (C# language reference): https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/arrays
- Microsoft Learn — `Array.Sort` method: https://learn.microsoft.com/en-us/dotnet/api/system.array.sort
- Microsoft Learn — `Array.BinarySearch` method: https://learn.microsoft.com/en-us/dotnet/api/system.array.binarysearch
- Microsoft Learn — `Array.IndexOf` and `Array.Reverse` methods: https://learn.microsoft.com/en-us/dotnet/api/system.array.indexof https://learn.microsoft.com/en-us/dotnet/api/system.array.reverse
- Microsoft Learn — Tuple types (tuple swap): https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/value-tuples
- Microsoft Learn — `Stopwatch` class: https://learn.microsoft.com/en-us/dotnet/api/system.diagnostics.stopwatch